

Plan de Branching

SmartLogix — Evaluación Parcial N°2 · DSY1106 Desarrollo Fullstack III

Mayo 2026

Contents

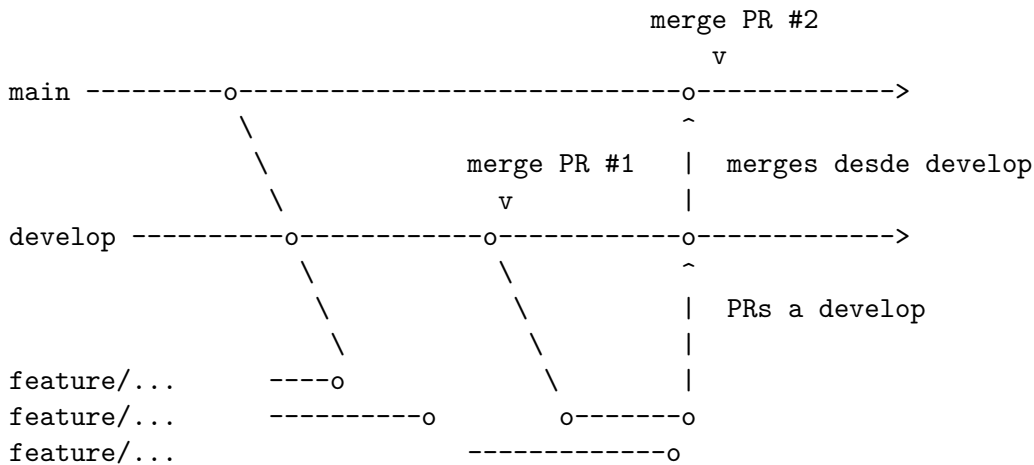
1. Estrategia adoptada: GitFlow simplificado	1
1.1 Diagrama del flujo	2
1.2 ¿Por qué GitFlow y no Trunk-Based?	2
2. Convenciones	3
2.1 Naming de ramas	3
2.2 Mensajes de commit	3
2.3 Pull Requests	3
3. Evidencia en el repositorio	4
3.1 Grafo de commits	4
3.2 Pull Requests cerrados	4
3.3 Ramas activas en <code>origin</code>	4
4. Gestión de conflictos	5
4.1 Tipos de conflictos enfrentados	5
a) Conflicto léxico en <code>README.md</code>	5
b) Conflicto de configuración en <code>docker-compose.yml</code>	5
c) Conflicto en <code>application.properties</code> de los MS	5
4.2 Protocolo de resolución	5
4.3 Lecciones aprendidas	5
5. Trabajo a futuro	6
6. Cumplimiento de los ítems 3 y 7 de la rúbrica	6

1. Estrategia adoptada: GitFlow simplificado

El proyecto SmartLogix utiliza una variante de **GitFlow** adaptada al tamaño del equipo (3–4 desarrolladores) y a la cadencia de la evaluación. Se eliminaron las ramas `release/*` y `hotfix/*` por innecesarias para un MVP académico, dejando un modelo de **tres tipos de rama**:

Rama	Propósito	Origen	Destino al cerrar
main	Código estable, entregable . Cada commit aquí es una versión que se puede defender.	—	—
develop	Rama de integración continua . Acumula features completas listas para release.	main	main (al consolidar entrega)
feature/<descripción>	Trabajo individual sobre una unidad concreta (un MS, una característica del frontend, un set de tests).	develop	develop vía Pull Request

1.1 Diagrama del flujo



1.2 ¿Por qué GitFlow y no Trunk-Based?

Criterio	GitFlow (elegido)	Trunk-Based
Tamaño del equipo	3–4 personas, varios features en paralelo. GitFlow permite features aisladas que no rompen main .	Mejor para equipos ≥ 10 con CI/CD maduro y feature flags.
Frecuencia de release	Una entrega académica formal (main = entregable).	Despliegues múltiples al día.
Riesgo de regresión	Bajo: develop actúa como red de seguridad antes de main .	Alto sin feature flags.
CI/CD	El proyecto no tiene pipeline de despliegue continuo, sólo build local.	Asume pipeline robusto.

Para el contexto (MVP académico, equipo pequeño, sin CI/CD remoto), GitFlow es la elección correcta.

2. Convenciones

2.1 Naming de ramas

`feature/<descripcion-corta-en-kebab-case>`

Ejemplos reales del repo:

- `feature/frontend-setup`
- `feature/inventario`
- `feature/improvement-pedido`
- `feature/solicitud_envio` (underscore por compatibilidad con tooling legacy)
- `feature/tests-unitarios`

2.2 Mensajes de commit

Se adopta **Conventional Commits**:

`<tipo>: <descripción imperativa en presente>`

`[cuerpo opcional con motivación]`

Tipos en uso:

- `feat:` — nueva funcionalidad
- `fix:` — corrección de bug
- `test:` — añadir o modificar tests
- `docs:` — sólo documentación
- `chore:` — tareas de infra/build
- `refactor:` — cambio interno sin alterar comportamiento

Ejemplos del historial real:

```
feat: dockerizar stack monorepo segun arquitectura del informe EV1
feat: completar microservicios + BFF con orquestación + READMEs
test: tests unitarios con Mockito + JaCoCo en los 3 microservicios
docs: justificar elección de Gradle frente al arquetipo Maven del PDF
feat: actuator + healthchecks reales en los 3 microservicios
```

2.3 Pull Requests

Cada `feature/*` se cierra con un PR contra `develop` (o `main`, si la rama de feature se cortó directo de `main` en la fase inicial):

- Título: descripción de qué se entrega
 - Descripción: contexto, cómo probar, screenshots si es UI
 - Reviewer: al menos 1 integrante distinto al autor
 - Merge strategy: **merge commit** (preserva la historia del feature, ítem visible en el grafo)
-

3. Evidencia en el repositorio

El historial real demuestra el flujo:

3.1 Grafo de commits

```
* 1f3256b (origin/main) Merge pull request #2 from smartLogixfs/feature/frontend-
setup
|\
| * d60e69b (origin/feature/frontend-setup) Actualizacion del frontend
| * 2096977 feat: dashboard visual terminado y estructura de rutas base
| * 0a3826b feat: setup frontend con typescript, bootstrap y api client
| | * b69a84a (origin/feature/inventario) Estructura microservicio inventario
| | /
| / |
* | 5dd5524 Merge pull request #1 from smartLogixfs/feature/improvement-pedido
|\ \
| / /
* | ec733d5 feat: initial archetype
| /
* 0f639b8 (origin/develop, main, develop) estructura inicial-1
* 8686396 (origin/feature/solicitud_envio) feat: dockerizar stack monorepo segun arquitectura
* d777a14 chore: initial monorepo commit
```

3.2 Pull Requests cerrados

Dos PRs mergeados con merge commit (visibles en el grafo):

PR	Rama de origen	Mergeada a	Hash del merge	Contenido
#1	feature/improvement-pedido	main	5dd5524	Mejoras de dominio en el MS de pedido (DTOs, validaciones, máquina de estados)
#2	feature/frontend-main setup	main	1f3256b	Setup inicial del frontend React 19 + Vite + dashboard

3.3 Ramas activas en origin

Al cierre de la entrega, el repositorio remoto contiene:

origin/main	# estable / entregable
origin/develop	# integración
origin/dev	# alias histórico (deprecado, no usar)
origin/feature/frontend-setup	# mergeada en PR #2
origin/feature/improvement-pedido	# mergeada en PR #1
origin/feature/inventario	# en revisión

<code>origin/feature/solicitud_envio</code>	<code># en revisión</code>
<code>origin/feature/tests-unitarios</code>	<code># tests unitarios + actuator + healthchecks</code>

4. Gestión de conflictos

4.1 Tipos de conflictos enfrentados

a) Conflicto léxico en `README.md`

Al mergear `feature/frontend-setup` con `main`, ambos branches habían editado la sección de “Estructura” del `README` raíz. Resolución manual: se eligió la versión más completa (la del `feature`) y se reconciliaron las descripciones de cada componente.

b) Conflicto de configuración en `docker-compose.yml`

Durante la integración de `feature/tests-unitarios`, otra rama (`feature/improvement-pedido`) había cambiado los `depends_on` simples por la forma larga con `condition: service_healthy`. El conflicto se resolvió **adoptando la forma larga** (más expresiva y compatible con `healthchecks` reales) y reconciliando ambas versiones.

c) Conflicto en `application.properties` de los MS

Al ejecutar los tests y agregar properties de Actuator desde dos ramas distintas, las claves `management.endpoint.health.*` se escribieron dos veces. Resolución: deduplicación manual + verificación de que Spring Boot toma sólo la última declarada.

4.2 Protocolo de resolución

1. `git fetch && git merge origin/develop` antes de empujar — para detectar conflictos localmente.
2. Si hay conflicto: abrir el archivo en VS Code / IntelliJ con la vista 3-way (current / incoming / base).
3. **Validar la solución compilando + corriendo tests** antes de marcar como resuelto (`git add + git commit`).
4. Documentar en el cuerpo del commit qué se resolvió: `merge: resolve conflict in docker-compose.yml (adopta service_healthy)`.
5. Push y notificar al equipo en el canal del proyecto.

4.3 Lecciones aprendidas

- **Branches cortas son mejores.** Las ramas que vivieron más de una semana acumularon más conflictos. La regla informal del equipo terminó siendo: **rebase con develop cada 2–3 días**.
 - **Commits atómicos.** Cuando un feature se commiteaba en chunks pequeños (un commit por archivo o por refactor), los conflictos eran triviales. Los commits gigantes con 20 archivos forzaban resolución manual lenta.
 - **`docker-compose.yml` y `README.md`** son los archivos con más conflictos potenciales por ser tocados desde múltiples features. Sugerencia futura: extraer fragmentos a archivos separados (compose con `include:`).
-

5. Trabajo a futuro

Mejoras a la estrategia que no se implementaron por tiempo:

1. **Protección de main** vía GitHub Branch Protection Rules (require PR + 1 review + status checks pasados).
 2. **CI/CD** que corra los tests JaCoCo + build de imágenes Docker en cada PR a **develop**.
 3. **Semantic Versioning** (v1.0.0, v1.1.0) etiquetando cada merge a **main**.
 4. **CODEOWNERS** para asignar reviewers automáticos por carpeta (**backend/microservices/*** → equipo backend, **frontend/*** → equipo frontend).
 5. **.gitmessage template** versionado en el repo para enforce de Conventional Commits desde el editor.
-

6. Cumplimiento de los ítems 3 y 7 de la rúbrica

Ítem 3 (5%): *“Implementa una estrategia de branching clara y organizada, utilizando Git para gestionar versiones de manera eficiente, con evidencia de merges, ramas y resolución de conflictos documentados”.*

- Estrategia documentada en §1 (GitFlow simplificado, modelo de 3 tipos de rama).
- Evidencia de **merges** en §3.2 (PRs #1 y #2 con sus hashes).
- Evidencia de **ramas múltiples** en §3.3 (8 ramas vigentes en origin).
- Evidencia de **resolución de conflictos** en §4 (tres casos concretos).

Ítem 7 (15%, defensa oral): *“Detalla la estrategia de branching utilizada, destacando cómo la estructura de ramas favoreció la colaboración...”.*

Material listo para defensa en §1.2 (justificación GitFlow vs Trunk-Based), §4.3 (lecciones de colaboración) y §5 (visión a futuro).